
DisasterAgentBench: An Evaluation Benchmark for Retrieval-Grounded Disaster Assessment with Drone-in-the-Loop Reinspection

Haifeng Wang

State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing
Wuhan University
Wuhan 430079, China
wanghaifeng68@whu.edu.cn

Wei He*

State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing
Wuhan University
Wuhan 430079, China
weihe1990@whu.edu.cn

Abstract

Existing agent benchmarks only weakly capture disaster assessment as a retrieval-grounded, map-grounded, and review-sensitive evaluation setting: a system must retrieve the correct event from a heterogeneous geospatial archive, ground its actions in the correct map source, localize evidence, decide whether drone reinspection is warranted, synthesize severity, and produce an auditable report. We present **DisasterAgentBench**, a benchmark and evaluation protocol for this workload, built from a disaster bundle containing 10,754 imagery layers, 298 events, 20 hazard categories, 563 GeoJSON overlays, and 258 label files. The benchmark defines six task families, structured task records, explicit tool budgets, deterministic grading rules, a gold-removed hidden-test package, and review-aware metrics. We also provide **DisasterAgent**, a traceable reference baseline that makes the benchmark executable end to end. Evaluation on a 262-task hidden-test package shows a consistent diagnostic pattern: tool-equipped systems can saturate retrieval, grounding, and localization, while assessment synthesis, calibration, and report grounding remain difficult. The resulting comparison indicates that correct geospatial interaction does not by itself guarantee correct disaster judgment, and that the benchmark remains sensitive to downstream assessment quality even when upstream tool use is nearly saturated. The contribution is therefore a reproducible benchmark and evaluation protocol that separates geospatial tool access from disaster judgment quality.

1 Introduction

Disaster assessment is neither a single-step perception task nor a generic tool-use task. A capable agent must retrieve the correct event from a large heterogeneous geospatial archive, ground decisions on the correct map source, localize affected regions, determine whether uncertain evidence warrants UAV reinspection, and produce an auditable report with calibrated uncertainty. Errors in any early stage can invalidate the full decision chain, while correct upstream interaction still may not imply a

*Corresponding author.

correct downstream assessment. We therefore study disaster assessment as a benchmark problem involving retrieval-grounded, map-grounded, drone-in-the-loop agent behavior.

Existing benchmarks evaluate broad reasoning, web interaction, software repair, or app-centric workflows, and they have substantially advanced the study of tool-using language agents [Liu et al., 2024, Mialon et al., 2024, Deng et al., 2023, Zhou et al., 2023, Xie et al., 2024, Jimenez et al., 2024, Yao et al., 2024, Trivedi et al., 2024]. However, they do not directly capture the coupled geospatial decision process required in disaster assessment: event retrieval from heterogeneous archives, map-source disambiguation, region-of-interest grounding, follow-up reinspection planning, and review-aware reporting. Conversely, traditional remote-sensing pipelines often assume fixed datasets, fixed models, or fixed interfaces, and therefore do not test whether an agent can manage the full decision loop.

Disaster assessment is a distinct agent problem for three reasons. First, correctness is event-conditioned: retrieving the wrong event or layer is a hard failure even if downstream reasoning is internally consistent. Second, evidence acquisition is adaptive rather than static: when map-side evidence is insufficient, the agent must decide whether and how to trigger UAV reinspection. Third, the setting is safety-sensitive: outputs should be uncertainty-aware, reviewable, and compatible with human override rather than optimized purely for autonomous completion.

To study this setting, we introduce **DisasterAgentBench**, a benchmark and evaluation protocol built from a curated disaster bundle spanning 10,754 imagery layers, 298 events, 20 hazard categories, 563 GeoJSON overlays, and 258 label files. DisasterAgentBench defines six task families—event retrieval, map grounding, damage localization, drone reinspection, assessment synthesis, and report generation—under explicit tool budgets, structured task records, deterministic grading rules, hidden-test packaging, and held-out transfer hooks. The main contribution is the benchmark itself: a reproducible evaluation setting for a class of agent workload that existing benchmarks only weakly cover.

To make this evaluation setting executable end to end, the benchmark includes a reference implementation, a UAV reinspection environment, and evidence-trace interfaces that expose intermediate state for scoring and audit. **DisasterAgent** instantiates the reference baseline within this substrate, coupling archive retrieval, map interaction, tiled localization, UAV mission planning, assessment synthesis, and report generation under an explicit workflow trace. These components are included because they make the benchmark runnable, auditable, and diagnostically useful, not because the paper’s primary claim depends on any one system component in isolation. More broadly, the benchmark turns post-disaster UAV reinspection into a shared evaluation substrate across multiple hazard and event scenarios, reducing the fragmentation that has made UAV-agent assessment difficult to compare across narrowly scoped setups.

The formal evaluation also motivates a benchmark-first reading rather than a simple architecture-ranking story. Tool-equipped systems saturate retrieval, grounding, and localization in the hidden-test run, but they differ sharply in drone execution, assessment synthesis, report grounding, calibration, latency, and traceability. In the present comparison, the completed systems occupy different operating points under the shared task definition and execution budget. DisasterAgentBench is useful precisely because it makes these tradeoffs measurable.

The paper is therefore organized around three benchmark design dimensions rather than around system-centric claims. First, the benchmark must capture event-grounded retrieval and map-grounded execution under realistic archive ambiguity. Second, it must expose long-horizon coordination demands rather than collapsing into one-shot perception or navigation. Third, it must evaluate safety-relevant assessment behavior, including reinspection decisions, uncertainty, and report grounding. The reference baseline and oracle controls are used only to operationalize these dimensions and to reveal which parts of the workflow are easy, which are brittle, and which remain the dominant bottlenecks under formal evaluation.

Our primary contribution is therefore not an application system, but an evaluation setting for a previously under-specified class of agent workloads. By turning disaster assessment into a benchmark with explicit task structure, evaluation protocol, execution constraints, and review-aware metrics, we aim to make this problem measurable, reproducible, and comparable across agent architectures. The central empirical question is correspondingly benchmark-centric: once systems can retrieve the

81 right event and operate the right map tools, does correct geospatial interaction actually translate into
82 correct disaster judgment?

83 **Contributions.** We make three contributions:

- 84 1. We introduce **DisasterAgentBench**, an evaluation benchmark for retrieval-grounded, map-
85 grounded, drone-in-the-loop disaster assessment, with six task families, structured task records,
86 explicit tool budgets, deterministic grading rules, and hidden-test packaging.
- 87 2. We provide the executable evaluation substrate for this benchmark, including **DisasterAgent**
88 as a traceable reference baseline and a post-disaster UAV reinspection environment that makes
89 reinspection tasks runnable, auditable, and comparable across hazard scenarios.
- 90 3. We present a reproducible evaluation protocol, hidden-test result, validation checks, and oracle
91 controls showing that correct archive and map interaction can saturate while assessment synthesis,
92 calibration, and report grounding remain the dominant bottlenecks.

93 2 Related Work

94 Our work sits at the intersection of language-agent architectures, realistic tool-use benchmarks, and
95 geospatial disaster analysis. Surveys and architectural frameworks such as CoALA and recent LLM-
96 agent surveys emphasize that strong agents require memory, planning, tool use, and environment
97 grounding rather than prompt-only reasoning [Wang et al., 2024, Xi et al., 2025, Sumers et al.,
98 2024]. Multi-agent frameworks such as AutoGen and Magentic-One further motivate explicit role
99 decomposition and managed handoffs when tasks mix retrieval, execution, and synthesis [Wu et al.,
100 2023, Fournay et al., 2024].

101 The closest prior evaluations come from realistic agent benchmarks. AgentBench and GAIA test
102 broad reasoning and tool use [Liu et al., 2024, Mialon et al., 2024]. Mind2Web, WebArena, WebLINX,
103 VisualWebArena, WorkArena, and OSWorld study grounded interaction in browsers and computer
104 environments [Deng et al., 2023, Zhou et al., 2023, Lu et al., 2024, Koh et al., 2024, Drouin et al.,
105 2024, Xie et al., 2024]. SWE-bench, τ -bench, and AppWorld evaluate software repair, tool-agent-user
106 interaction, and app-centric workflows [Jimenez et al., 2024, Yao et al., 2024, Trivedi et al., 2024].
107 These benchmarks are invaluable, but they do not directly capture the geospatial retrieval, map-source
108 ambiguity, and audit-oriented reporting required in disaster assessment.

109 Our benchmark is also related to remote sensing and disaster mapping pipelines, but the problem
110 formulation is different. Traditional pipelines typically assume a fixed dataset, a fixed model, or
111 a fixed user interface. DisasterAgentBench instead asks whether a tool-using language agent can
112 solve the full decision chain: retrieve the right event, activate the right map source, localize relevant
113 evidence, synthesize a severity judgment, and expose that judgment in a reviewable report.

114 3 Task Setting and Benchmark Construction

115 3.1 Problem Formulation

116 We formalize end-to-end disaster assessment as follows. Let x be a user request, $\mathcal{A}$ a geospatial
117 archive of indexed assets, $\mathcal{T}$ the allowed tool set, and $\tau = (a_1, \dots, a_T)$ an agent trajectory of tool
118 calls and intermediate observations. The agent must predict a structured output

$$y = (e, m, g, u, s, c, r),$$

119 where e is an event identifier, m is a map source or region-of-interest grounding decision, g is an
120 optional geospatial output such as GeoJSON, u is an optional drone reinspection plan, s is a severity
121 label, c is a calibrated confidence value, and r is a structured report. Different benchmark tasks
122 supervise different subsets of y , but all tasks preserve the same execution setting: archive retrieval,
123 tool use, drone-aware follow-up planning, and evidence-grounded evaluation.

124 3.2 Why Disaster Assessment Is a Distinct Agent Problem

125 We define end-to-end disaster assessment as the following problem. Given a user request, an agent
126 must (i) identify the relevant event and data assets, (ii) activate an appropriate map source or overlay,

(iii) localize the target region or damaged objects, (iv) aggregate retrieved and detected evidence into an assessment, and (v) produce a structured report with explicit confidence. This workflow differs from standard web navigation because the target state is not merely an interaction trace; it is a grounded geospatial assessment whose correctness depends on retrieval, localization, synthesis, and auditability.

3.3 Source Bundle

Our benchmark is built from a local disaster bundle containing indexed geospatial layers and event metadata. Corpus-scale quantities are summarized in Appendix Table 2, while split and hidden-test composition are reported separately in Appendix D. At the corpus level, the bundle is large enough to introduce realistic retrieval ambiguity, event diversity, and heterogeneous asset types, while still allowing event-balanced task construction.

Archive structure. The indexed source bundle mixes event-centric disaster areas, label files, and extracted project metadata. At the top-group level, the bundle contains 10,490 `disaster_area` files, 258 `labels` files, and a small number of project-level metadata files. Only 36 events expose both area and label artifacts, while 263 are area-only and 5 are label-only. In the raw archive, ‘`disaster_area`’ files are heavily skewed toward SAR-style records (10,184 files under a single `sar_data` event), which means that a benchmark built directly by file frequency would be severely imbalanced. DisasterAgentBench is therefore designed to be *event-balanced* rather than file-balanced: the unit of sampling is the event-task pair, not the raw file.

3.4 Task Record and Task Families

Each benchmark item is represented as a structured task record with the fields summarized in the accompanying JSON schema: task identifier, split, task family, user request, allowed tools, tool budget, context, gold output, evaluation specification, and metadata. This design has two goals. First, it allows all baselines to be run under a uniform tool budget. Second, it makes grading deterministic for tasks whose success depends on multiple partial outputs.

We currently define six core task families:

1. **Event retrieval:** identify the correct event and relevant assets from a natural-language request.
2. **Map grounding:** switch to the correct tile source, overlay, or region of interest.
3. **Damage localization:** generate geospatial outputs that localize affected areas or objects.
4. **Drone reinspection:** choose and parameterize a follow-up UAV mission over uncertain or high-impact regions.
5. **Assessment synthesis:** infer severity, impact summary, and confidence from retrieved and visual evidence.
6. **Report generation:** produce a structured analyst-facing report grounded in evidence.

Unlike generic tool-use benchmarks, drone tasks are explicitly stateful. A drone task record therefore specifies not only the target region but also a simulator start state, an admissible mission family, acceptable interrupt actions, and a success condition over route completion or ROI coverage. This lets the benchmark distinguish between merely mentioning a UAV and proposing an operationally plausible reinspection workflow.

3.5 Annotation Protocol and Splits

We use an event-centric annotation protocol. For each task, annotators first select the target event, then specify one or more valid user requests, the permissible tool set, the target region or map source, and the gold outputs needed for deterministic evaluation. Assessment and report tasks also require evidence keys that specify the minimum facts or artifacts a valid answer must reference. Drone reinspection tasks additionally annotate a valid mission family, a route or coverage target, and whether pause, resume, or return-home actions would be acceptable under operator intervention. This protocol prevents benchmark success from collapsing into template matching on report text.

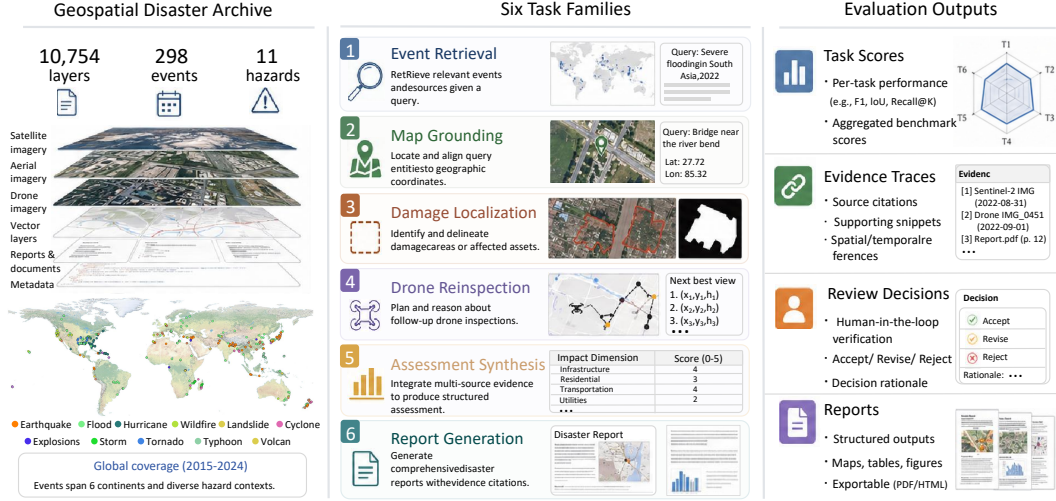


Figure 1: DisasterAgentBench overview. The benchmark is built on a geospatial disaster archive with 10,754 files, 298 events, and 20 hazard types, and defines six task families evaluated through task scores, evidence traces, review decisions, and reports.

To avoid leakage between train, development, and evaluation, we use event-level splits. We also include hazard-held-out, region-held-out, and time-held-out settings to probe transfer. The benchmark includes a public development set and a hidden test set, and the split manifest is distributed with the benchmark metadata.

4 Reference Baseline: DisasterAgent

4.1 System Overview

DisasterAgent is a retrieval-grounded multi-agent framework designed to convert a high-level disaster request into geospatially grounded evidence and a structured assessment. The system comprises:

- a **Data Agent** for indexed archive retrieval,
- a **Map Agent** for map source switching and overlay publication,
- a **Model Agent** for tiled real-time or background localization,
- a **Drone Agent** for simulated aerial reinspection under mission and operator-control constraints,
- an **Assessment Agent** for evidence synthesis and severity estimation,
- a **Report Agent** for report rendering,
- and an **Orchestration Agent** that routes and traces the workflow.

4.2 Multi-Agent Task Orchestration

DisasterAgent uses explicit workflow routing rather than an unconstrained planner. The orchestration layer selects among a small set of routes that cover retrieval, map preparation, localization, optional reinspection, assessment, and reporting. We represent a workflow as a directed acyclic graph

$$W = (N, D),$$

where nodes N are agent-level stages and edges D encode execution dependencies. A lightweight workflow bridge carries the upstream intent and routing decision across stages so that the benchmark can audit handoffs rather than only final outputs.

196 4.3 Retrieval-Grounded Initialization

197 The Data Agent searches a local indexed bundle of disaster assets and returns candidate map layers,
 198 overlays, or files associated with the requested hazard and event. Search operates over indexed
 199 metadata such as disaster type, event name, and file path, with query normalization for common
 200 hazard aliases. This stage matters because benchmark correctness is event-conditioned: a correct
 201 localization or report built on the wrong event is still a hard failure.

202 4.4 Geospatial Tool Use

203 After retrieval, the system prepares a displayable target, switches to a reachable tile source, publishes
 204 GeoJSON overlays, and passes the resulting context to the Model Agent. The Model Agent then
 205 performs tiled detection over either the present view or a larger background extent. For the benchmark,
 206 the key distinction is between *where to look* and *what to detect*.

207 4.5 Drone Reinspection Execution Substrate

208 To make drone tasks executable inside the benchmark, the Drone Agent runs in a simplified but
 209 stateful reinspection substrate for uncertain or high-impact regions. The simulator supports multiple
 210 mission families and explicit control interrupts such as pause, resume, stop, and return-home. Its role
 211 is benchmark-facing rather than aerodynamics-facing: it lets the evaluation score mission selection,
 212 region coverage, interruption handling, and operator escalation instead of treating UAV follow-up as
 213 a purely textual suggestion.

214 4.6 Assessment Synthesis

215 The Assessment Agent converts retrieved and visual evidence into a structured output with a severity
 216 label, confidence, metrics, recommendations, and an assessment overlay. Because this stage remains
 217 the least reliable part of the baseline, the benchmark pairs it with an explicit evidence-trace scaffold
 218 rather than treating the final assessment as an opaque score.

219 4.7 Evidence-Trace Scaffold via HEG

220 In the baseline, we use a lightweight **Hierarchical Evidence Graph (HEG)**-style scaffold to link
 221 retrieved assets, map views, detections, temporal comparisons, and review decisions. Its role here
 222 is pragmatic: it gives the benchmark an explicit evidence interface so that assessment and report
 223 tasks can be audited, graded, and ablated against a structured trace rather than a single opaque score.
 224 Let $G = (V, E)$ denote this evidence graph, where each node $v \in V$ has a type $t(v)$ and quality
 225 score $q(v)$, and each edge $(u, v) \in E$ has a relation type such as spatial overlap, temporal match, or
 226 support/conflict. We score severity class k as

$$S_k = \sum_{v \in V} \alpha_{t(v)} q(v) \phi_k(v) + \sum_{(u,v) \in E} \beta_{r(u,v)} \psi_k(u, v),$$

227 where ϕ_k scores node-level support for class k and ψ_k scores pairwise consistency. The final severity
 228 prediction is $\hat{s} = \arg \max_k S_k$. We then compute a confidence score from graph agreement, conflict,
 229 and coverage:

$$c = \sigma(\gamma_1 \text{agreement}(G) - \gamma_2 \text{conflict}(G) - \gamma_3 \text{coverage_gap}(G)).$$

230 Within the benchmark, this scaffold provides severity estimation, calibrated confidence, and an
 231 explicit explanation path for analysts.

232 4.8 Risk-Aware Review and Reporting

233 Not every disaster task should be treated as fully autonomous. We therefore use a risk-aware review
 234 policy that routes uncertain or high-impact outputs to human review. A simple decision rule is

$$\text{review} = \mathbb{I}[c < \tau_c \vee (1 - c) \cdot \text{impact} > \tau_r],$$

235 where c is the calibrated confidence, impact is a task-level risk proxy, and τ_c, τ_r are thresholds chosen
 236 on development data. The Report Agent renders the final structured output from the assessment trace,
 237 preserving severity, evidence, recommendations, and raw artifacts for audit.

238 5 Evaluation Protocol

239 5.1 Benchmark Instantiation

240 We instantiate DisasterAgentBench from the local indexed disaster bundle by sampling event-
241 conditioned tasks rather than raw files. Each benchmark item specifies a user request, a task family, a
242 split, an allowed tool set, and one or more gold targets. The source bundle exposes 298 unique event
243 names across 20 hazard categories, but because several event names recur across different hazards,
244 the split manifest is defined over 304 *hazard-event* units rather than 298 bare event-name strings.
245 Supervision is asymmetric: only 36 units provide paired `disaster_area` and `labels` artifacts,
246 whereas 263 are area-only and 5 are label-only. This asymmetry motivates family-specific task
247 construction rather than a single uniform supervision recipe. The split manifest contains 212 train
248 units, 46 development units, and 46 hidden-test units. The hidden-test package materializes these
249 units into 262 gold-removed task records. That is, the hidden-test split is released as task instances
250 with inputs and metadata but with gold outputs withheld for evaluation. Appendix Table 8 makes the
251 counting policy explicit by separating families whose candidate pools are already derivable from the
252 manifest from those that still depend on frozen severity or evidence annotations.

253 The split is broad enough to support nontrivial generalization claims even before the final task records
254 are frozen. Training currently covers all 20 hazard categories represented in the bundle, while the
255 public development and hidden-test manifests each cover 15 hazards. The largest hazard families
256 are approximately stratified across splits: flooding contributes 45/10/10 train/dev/hidden-test units,
257 Hurricane contributes 35/7/7, and Earthquake contributes 23/5/5. Split-level supervision composition
258 is summarized in Appendix Table 7; importantly, paired area+label units remain rare in every split, so
259 evaluation should not assume dense supervision outside a small subset of events.

260 5.2 Task Instantiation and Difficulty Control

261 The benchmark is instantiated from *hazard-event* units rather than raw files. Event-retrieval and map-
262 grounding tasks require retrievable and displayable assets; localization tasks require geometry from
263 `disaster_area` or `labels`; drone tasks require an ROI plus an executable mission template; assess-
264 ment and report tasks require severity targets and evidence keys. We use explicit *easy/medium/hard*
265 buckets defined by retrieval ambiguity, map-source ambiguity, spatial scale, temporal complexity,
266 and review requirement, so that the benchmark remains balanced across families and difficulty levels
267 rather than being dominated by easy retrieval items.

268 5.3 Evaluation Settings and Leakage Control

269 We use event-level train/dev/test partitioning as the default split rule, so that all artifacts associ-
270 ated with the same disaster event remain in a single split. In practice, the split manifest uses
271 hazard-stratified hazard-event units, deterministic hash ordering within each hazard, and minimum
272 development/hidden-test allocations for hazards with at least four events. This is especially important
273 because file-level splitting would be dominated by the `sar_data` subtree, which contributes 10,184
274 files under a single event identifier and would otherwise swamp the benchmark. In addition to the
275 default split, the benchmark exposes hazard-held-out, region-held-out, and time-held-out settings
276 for transfer analysis. Hidden-test evaluation is distributed as task records without gold outputs. This
277 protocol is intended to prevent leakage through repeated event identifiers, overlapping pre/post assets,
278 or duplicated geospatial extents, while preserving event-balanced evaluation despite severe file-level
279 skew.

280 We use four evaluation settings: **Event-IID** for the main table, plus **hazard-held-out**, **region-held-**
281 **out**, and **time-held-out** for transfer. All prompts, routing rules, and review thresholds are tuned
282 once on the public Event-IID development split and then frozen for all hidden-test and held-out
283 evaluations.

284 5.4 Comparison Systems

285 Our target comparison set under matched tool budgets is:

- 286 1. a direct LLM baseline without external tools,

- 287 2. a single-agent ReAct-style tool user [Yao et al., 2023],
- 288 3. a retrieval-plus-detector pipeline without explicit orchestration,
- 289 4. a single-agent system with the full tool set but no explicit workflow routing,
- 290 5. an oracle-retrieval variant that receives the correct event context but still performs downstream
- 291 map, detection, and assessment steps,
- 292 6. the full DisasterAgent system,
- 293 7. and a human-reference workflow on a stratified subset of tasks.

294 This comparison set separates retrieval errors from downstream orchestration and assessment errors;
 295 the oracle-retrieval variant is particularly useful for measuring how much difficulty remains after
 296 event grounding is solved. The hidden-test table in Section 6 includes the full comparison set, but
 297 only the four tool-equipped systems have completed adjudicated runs under the frozen evaluation
 298 protocol; the direct-LLM, retrieval-plus-detector, and human-reference controls are therefore shown
 299 as intentionally unpopulated rows.

300 5.5 Benchmark Validation Protocol

301 Because DisasterAgentBench is positioned as an Evaluations & Datasets contribution, benchmark
 302 validation must precede model interpretation. The validation suite reports coverage and diversity,
 303 agreement on a dual-annotated subset, oracle execution validity, human/reference solvability on
 304 a stratified subset, and ranking stability under resampling or repeated runs. Appendix 3 records
 305 the validation snapshot. The human-annotation files now enter that snapshot directly: the 44-task
 306 `assessment_synthesis` subset has overlapping A1/A2 labels for dual-annotator agreement, while
 307 the remaining families are covered by single-annotator checks against frozen references. The formal
 308 oracle rows, by contrast, come from hidden-test executions. Quantities that have not yet been
 309 populated under the boundary remain blank rather than being estimated from earlier traces.

310 5.6 Metrics and Statistical Protocol

311 We group metrics into four levels:

- 312 • **Benchmark-quality metrics:** execution-validity rate, annotation agreement, human/reference
 313 success, and ranking stability.
- 314 • **Task metrics:** top-1 event accuracy, top- k recall, grounding success, center distance, GeoJSON
 315 F1/IoU, mission validity, ROI coverage, intervention success, severity F1, and report grounding
 316 score.
- 317 • **System metrics:** end-to-end success, latency, tool calls, and runtime cost.
- 318 • **Risk metrics:** ECE, review rate, false-autonomy rate, and unnecessary-review rate.

319 For drone tasks, we separate *mission validity* from *geometric coverage*. If P denotes the planned or
 320 executed flight path and R denotes the target reinspection region, we score ROI coverage as

$$\text{coverage}(P, R) = \frac{\text{area}(\text{buffer}(P, \rho) \cap R)}{\text{area}(R)},$$

321 where ρ is a fixed visibility radius derived from altitude and sensor assumptions. We report interven-
 322 tion compliance separately by checking whether the system preserves or correctly executes required
 323 pause, resume, stop, or return-home actions under the task specification.

324 **Partial credit.** Some tasks admit partial success. For example, a system may retrieve the correct
 325 event but fail to select the optimal map source, or it may localize the correct region while underesti-
 326 mating severity. The task schema therefore stores both a primary metric and optional thresholded
 327 grading rules such as center-distance thresholds, minimum IoU, or minimum ROI coverage.

328 **Statistical protocol.** All stochastic systems should be run with at least three seeds. Main tables
 329 should report means with 95% bootstrap confidence intervals, and main pairwise claims should use
 330 paired bootstrap or approximate randomization on hidden-test predictions. Review and calibration
 331 thresholds must be fixed on development data before hidden-test evaluation.

5.7 Implementation and Execution Protocol

All baselines run against the same indexed archive, map interface, tiled model backend, and drone simulator. Every run logs the tool trace, intermediate artifacts, runtime, and final report. We also report *infrastructure-induced failure* separately from reasoning failure so that backend outages are not misread as cognitive agent errors. Appendix H converts this setup into a concrete freeze-run-score workflow, including which subsets populate each table, which variants require three seeds, and which artifacts must be saved for audit.

6 Results

We report the hidden-test comparison on the 262-task Event-IID hidden-test package using the frozen evaluation configuration. Table 1 summarizes the completed runs for the comparison systems and oracle controls under the shared benchmark setting. The single-agent and oracle baselines are averaged over three seeds, while the full-DisasterAgent row is populated from the execution budget used for the benchmark. Additional robustness, review, ablation, and trace-diagnostic panels remain in Appendix A.

Throughout this section, we distinguish *execution success* from *task success*. Execution success means that a run completes without infrastructure failure, policy interruption, or scorer-level invalidation; task success means that the task meets its declared primary metric threshold and therefore contributes positively to end-to-end benchmark success. This distinction matters because a system may execute the full workflow and still fail the benchmark on semantic assessment or report grounding.

Table 1: Hidden-test comparison. Columns report the family-level primary metrics together with end-to-end success, calibration, latency, and tool use on the 262-task hidden-test split under the frozen evaluation configuration. Single-agent and oracle baselines are means over three seeds; the full-DisasterAgent row is populated from the execution budget used for the benchmark. Empty cells denote comparison conditions that remain unpopulated under the frozen boundary.

System	Retr.	Ground.	Local.	Drone	Assess.	Report	E2E	ECE	Latency	Tools
Single-agent ReAct	1.000	1.000	1.000	0.000	0.500	0.278	0.588	0.285	0.106	5.511
Single-agent full tools	1.000	1.000	1.000	1.000	0.500	0.278	0.748	0.199	0.115	5.000
Oracle grounding single-agent	1.000	1.000	1.000	0.000	1.000	0.278	0.672	0.050	0.105	4.511
Oracle retrieval single-agent	1.000	1.000	1.000	0.000	0.500	0.278	0.588	0.269	0.105	4.511
Oracle all gold	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.010	0.000	0.000
Full DisasterAgent	1.000	1.000	1.000	1.000	0.114	0.222	0.683	0.686	3.970	2.008

The main result is benchmark-centric. First, explicit access to the full geospatial tool chain is sufficient to saturate retrieval, grounding, and localization: every completed system reaches 1.000 on those upstream families. Second, the supporting oracle rows show that the remaining difficulty is structured rather than monolithic: improving access to correct event context can raise downstream performance, but does not by itself eliminate assessment, reporting, or execution bottlenecks. Third, the two baselines without a viable drone path collapse to 0 on the drone family and accumulate infrastructure-failure mass on those blocked trajectories, confirming that executable reinspection is a substantive part of the benchmark rather than a cosmetic tool hook.

Taken together, these results clarify what DisasterAgentBench measures. The benchmark is not limited by archive lookup or basic map interaction; instead, it remains sensitive to the harder problem of converting correct geospatial interaction into correct, calibrated, and analyst-ready disaster judgment. The non-oracle systems differ in end-to-end score, latency, calibration, and report grounding, but the more important observation is that upstream competence does not guarantee downstream assessment quality. In this sense, the comparison is useful because it separates *tool access* from *decision quality*, and also *execution efficiency* from *workflow governability*, under a shared task definition and matched execution constraints. Appendix Figure 2 visualizes this upstream-versus-downstream gap directly.

7 Limitations and Broader Impact

The benchmark is derived from a locally curated disaster bundle and will likely inherit collection biases across hazard types, regions, and annotation styles. Some hazards may be overrepresented, while others may only appear in weakly labeled form. The UAV environment, while more realistic than a static waypoint player, is still not a full physical or regulatory simulator; it models operational

decision variables such as mission mode, route, pause/resume, and return-home, but not full weather, battery chemistry, or communications failure. The DisasterAgent result should therefore be read as a reference point for the present implementation and evaluation budget, not as a ceiling on structured multi-agent orchestration. The present validation snapshot, while grounded in provided human annotations, still falls short of a fully adjudicated multi-family human study because only the assessment subset has dual annotation. Moreover, agent evaluation in disaster settings is safety-critical: overconfident outputs can mislead analysts, while underconfident systems may increase review burden and latency. We therefore recommend that all benchmark tasks preserve evidence traces, explicit uncertainty fields, and clear disclaimers against fully autonomous operational use.

Release scope. The benchmark distribution exposes the split-manifest metadata, task schema, frozen evaluation configuration, hidden-test package, scorer-facing task records, and aggregated run files needed to reproduce reported tables. The underlying local disaster bundle is not treated as an automatically redistributable corpus asset; where licenses or source restrictions prevent direct redistribution, the distribution boundary falls back to task-level metadata, frozen prompts and thresholds, and evaluator-side prediction and trace files.

8 Conclusion

Disaster assessment exposes a class of agent workload that benchmarks only weakly cover: event-grounded retrieval over large geospatial archives, map-source disambiguation, localized damage analysis, drone-assisted follow-up, and uncertainty-aware reporting. The main contribution of this paper is **DisasterAgentBench**, a benchmark and evaluation protocol for this setting. The benchmark is accompanied by an executable reference substrate, including a baseline implementation, a UAV environment, and evidence-trace interfaces that make reinspection-aware and review-aware evaluation operational rather than purely conceptual. By grounding this evaluation substrate in multi-hazard, multi-event disaster data, the benchmark provides a shared post-disaster UAV-agent setting in which mission selection, route execution, interruption handling, and evidence-grounded follow-up can be compared under one formal task definition. The benchmark now includes a 262-task gold-removed hidden-test package, a frozen evaluation configuration, and an initial hidden-test result. The main empirical takeaway is that correct geospatial interaction is not yet equivalent to correct disaster judgment, and that this gap remains measurable even when upstream retrieval and grounding are nearly saturated. By packaging task families, split design, grading rules, and a concrete freeze-run-score workflow, we aim to make this benchmark class reproducible, diagnosable, and comparable across agent architectures.

References

- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL <https://openreview.net/forum?id=kiYqb03wqw>.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR, 2024. URL <https://proceedings.mlr.press/v235/drouin24a.html>.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-one: A generalist multi-agent system for solving complex tasks, 2024. URL <https://arxiv.org/abs/2411.04468>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.

- 428 Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham
429 Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating
430 multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting
431 of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905.
432 Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.50. URL
433 <https://aclanthology.org/2024.acl-long.50/>.
- 434 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
435 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui
436 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie
437 Tang. Agentbench: Evaluating llms as agents. In *International Conference on Learning Repre-
438 sentations*, 2024. URL [https://proceedings.iclr.cc/paper_files/paper/2024/hash/
439 e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html](https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html).
- 440 Xing Han Lu, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with
441 multi-turn dialogue. In *Proceedings of the 41st International Conference on Machine Learning*,
442 volume 235 of *Proceedings of Machine Learning Research*, pages 33007–33056. PMLR, 2024.
443 URL <https://proceedings.mlr.press/v235/lu24e.html>.
- 444 Grégoire Mialon, Clémentine Fourier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a
445 benchmark for general AI assistants. In *International Conference on Learning Representations*,
446 2024. URL <https://openreview.net/forum?id=fibxvahvs3>.
- 447 Theodore R. Sumers, Shunyu Yao, Karthik R. Narasimhan, and Thomas L. Griffiths. Cognitive
448 architectures for language agents. *Transactions on Machine Learning Research*, 2024, 2024. URL
449 <https://openreview.net/forum?id=1i6ZCvflQJ>.
- 450 Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank
451 Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps
452 and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting
453 of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076.
454 Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.850. URL
455 <https://aclanthology.org/2024.acl-long.850/>.
- 456 Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai
457 Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large
458 language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
459 doi: 10.1007/s11704-024-40231-1. URL [https://link.springer.com/article/10.1007/
460 s11704-024-40231-1](https://link.springer.com/article/10.1007/s11704-024-40231-1).
- 461 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li,
462 Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via
463 multi-agent conversation framework, 2023. URL <https://arxiv.org/abs/2308.08155>.
- 464 Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe
465 Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou,
466 Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongx-
467 iang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing
468 Huang, and Tao Gui. The rise and potential of large language model based agents: a survey.
469 *Science China Information Sciences*, 68(2):121101, 2025. doi: 10.1007/s11432-024-4222-0. URL
470 <https://link.springer.com/article/10.1007/s11432-024-4222-0>.
- 471 Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua,
472 Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese,
473 Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-
474 ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- 475 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao.
476 React: Synergizing reasoning and acting in language models. In *International Conference on
477 Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

478 Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. τ -bench: A benchmark for
 479 tool-agent-user interaction in real-world domains, 2024. URL [https://arxiv.org/abs/2406.](https://arxiv.org/abs/2406.12045)
 480 12045.

481 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
 482 Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment
 483 for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.

484 A Formal Evaluation Supplement

485 This appendix contains supplementary formal-evaluation material that supports the main benchmark
 486 claim without consuming main-paper pages. It records frozen protocol components, supplementary
 487 evaluation slices, and diagnostic artifacts that complement the main hidden-test result.

Table 2: Corpus-scale benchmark statistics from the indexed disaster bundle. Split composition and hidden-test counts are reported separately from these source-bundle totals.

Statistic	Value
Imagery layers	10,754
Hazard categories	20
Event identifiers	298
GeoJSON overlays	563
Label files	258
Hazard-event split units	304
Paired area+label events	36
Area-only events	263
Label-only events	5
Task families	6

Table 3: Benchmark-validation snapshot. The human-annotation files support two distinct checks: dual-annotator agreement on the `assessment_synthesis` subset, and broader single-annotator consistency checks against frozen references on the remaining families. Empty cells indicate validations reserved in the formal protocol but not yet populated by adjudicated runs.

Validation quantity	Value	Notes
Hidden-test task coverage	262 tasks / 6 families	Event-IID hidden-test package
Hazard coverage in split manifest	15 hidden-test hazards	split-level statistic
Human-label coverage	306 provided labels / 262 task instances	assessment has dual annotation; the other families currently have single-annotator coverage
Dual-annotated subset	44 assessment tasks	provided A1/A2 labels with task-level overlap only on the assessment family
Assessment agreement	0.432 severity agreement	Cohen’s $\kappa = 0.210$ and evidence F1 1.000 on the dual-annotated assessment subset
Oracle execution validity	oracle-all-gold E2E 1.000	formal 3-seed oracle validation on all 262 tasks
Single-annotator reference checks	5 remaining families + assessment pass	broader family coverage uses one provided annotator checked against frozen references
Ranking stability	12 grouped systems / 66 pairwise tests	3-seed aggregation over the main-table systems and six formal ablations

System	Upstream execution profile	Downstream synthesis profile
Single-agent ReAct	0.750	0.389
Single-agent full tools	1.000	0.389
Oracle retrieval single-agent	0.750	0.389
Oracle grounding single-agent	0.750	0.639
Oracle all gold	1.000	1.000
Full DisasterAgent	1.000	0.168

Figure 2: Benchmark takeaway profile on the hidden test. The upstream score is the mean of retrieval, grounding, localization, and drone metrics; the downstream score is the mean of assessment and report metrics. The gap between the two columns visualizes the main claim of the benchmark: correct archive and map interaction do not imply correct synthesis or report grounding.

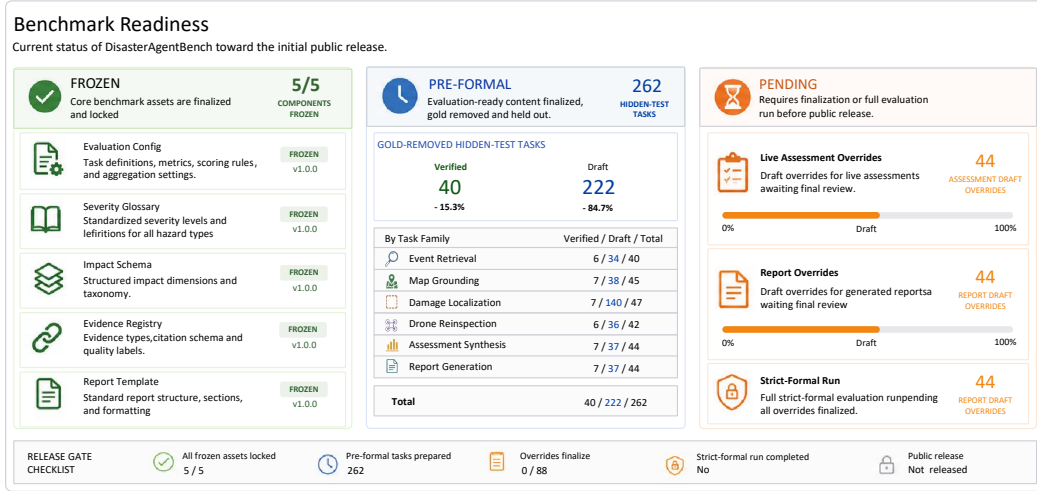


Figure 3: Benchmark formalization ledger. The figure summarizes the frozen infrastructure, adjudication boundary, and evaluation state transitions that define the formal benchmark used for hidden-test scoring.

B Trace-Grounded Prototype Diagnostics

We classify failures into wrong event retrieval, wrong map-source grounding, localization miss, wrong reinspection mode or route, temporal mismatch, conflicting evidence, and overconfident report generation. Figure 5 instantiates this diagnostic view using representative traces selected to complement the formal hidden-test summary with trace-level attribution.

C Supplementary Benchmark Context

This appendix records supplementary benchmark context, including split metadata, family composition, frozen policy files, and reproducibility-oriented run structure. These details are included to support inspection without interrupting the main-paper narrative.

D Split Manifest Summary

The split manifest preserves broad hazard coverage while still reflecting the underlying supervision asymmetry of the source bundle. Across all 304 units, the largest hazard families are flooding (65), Hurricane (49), and Earthquake (33). The hidden-test split remains intentionally diverse despite its small size, with its largest families being flooding (10), Hurricane (7), and Earthquake (5), followed by several hazards with one to three events each. This is sufficient for a first benchmark version, but the paper should still report per-family task counts after task instantiation is fixed.

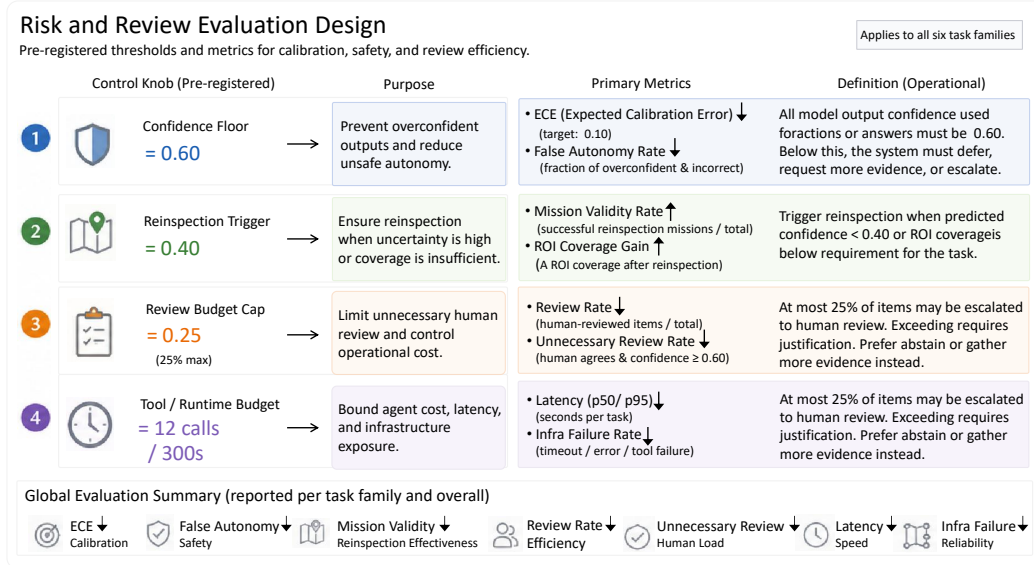


Figure 4: Frozen risk and review quantities used by the formal evaluation protocol. The panel functions as a threshold register for supplementary review analysis rather than as a post-hoc fitted curve.

Table 4: Supplementary robustness and orchestration results. Held-out rows use the materialized supplementary transfer manifests; Full DA in these rows uses the fast bounded evaluation adapter after the non-fast bounded path timed out on the hazard slice.

Evaluation slice	SA full tools	Full DA	Infra fail	Status
Hazard-held-out	0.690	0.690	0.000	completed on 29-task Earthquake slice
Region-held-out	0.714	0.714	0.000	completed on 77-task Asia-Pacific slice
Time-held-out	0.779	0.779	0.000	completed on 77-task post-2023 slice
Long-horizon	0.492	0.362	0.000	completed formal slice on 130 drone/assessment/report tasks

E Task-Family Composition and Eligibility Policy

This table makes explicit which supervision file anchors each task family. It is useful for understanding why the benchmark separates geometry-centric performance from assessment/report performance and why the latter families depend on additional evidence and slot-level constraints.

F Execution Artifacts

The repository includes several benchmark assets that are useful for converting the draft into a fully evaluated submission:

- a split-manifest file with 212 train units, 46 development units, and 46 hidden-test units;
- workflow-trace summaries covering completed display, assessment, and mixed retrieval-assessment executions together with a small number of failed detection traces;
- assessment summaries confirming that the pipeline emits disaster-type and severity labels before formal scoring;

Table 5: Supplementary drone and review results under the same hidden-test execution boundary as the main table. Oracle and main-system reference rows are populated from the frozen runs together with completed review and drone ablations.

Variant	Mission	ROI	Assess.	ECE	Review	E2E
Oracle all gold	1.000	1.000	1.000	0.010	0.000	1.000
Oracle grounding single-agent	0.000	1.000	1.000	0.050	0.000	0.672
Full DisasterAgent	1.000	1.000	0.114	0.686	0.000	0.683
w/o Drone Agent	0.000	1.000	0.500	0.195	0.786	0.588
w/o Review Policy	1.000	1.000	0.500	0.179	0.000	0.748
w/o Drone + Review	0.000	1.000	0.500	0.097	0.000	0.588

Table 6: Supplementary ablation results under the same hidden-test execution boundary as the main table. Rows report completed component removals and the matched single-agent replacement baseline.

Variant	E2E	Assess.	Report	Interpretation
Oracle all gold	1.000	1.000	1.000	oracle ceiling under the frozen scorer
Oracle grounding single-agent	0.672	1.000	0.278	removes assessment ambiguity but not drone/report bottlenecks
Full DisasterAgent	0.683	0.114	0.222	multi-agent reference
w/o Data Agent	0.733	0.500	0.278	removes retrieval-specialized delegation while preserving downstream structure
w/o Model Agent	0.752	0.523	0.222	removes model-side perception support; report grounding remains the main loss
w/o Drone Agent	0.588	0.500	0.278	collapses the drone family and lowers end-to-end completion
w/o Review Loop	0.748	0.500	0.278	preserves task score but suppresses review-triggered governance
w/o Workflow Bridge	0.748	0.500	0.278	preserves task score while increasing review activity and tool overhead
w/o HEG	0.748	0.500	0.278	preserves task score while increasing tool calls and review pressure
Single-agent replacement	0.748	0.500	0.278	populated by the formal single-agent full-tools run

- a repository-local freeze package containing the severity glossary, impact-fact format, evidence-key registry, report template, and package manifest;
- a hidden-test package containing the frozen evaluation configuration, hidden-test manifest meta-data, and 262 gold-removed task records.

These files should not be interpreted as formal system-performance results. Instead, they serve as benchmark assets, qualitative case-study evidence, and sanity checks on the execution pipeline.

G Severity and Evidence Freeze Protocol

Assessment and report tasks are structurally present in the hidden-test package, with 44 task records per family. Assessment summaries also confirm that the pipeline emits disaster type and severity labels before formal scoring, with observed labels spanning multiple disaster types and severity levels. These summaries are diagnostic artifacts rather than benchmark gold. The repository includes a frozen core-policy package together with adjudication-ready task-level assessment and report overrides, which make the final hidden-test evaluation possible.

With adjudicated override rows in place, assessment and report tasks can be scored under the same formal hidden-test protocol as the rest of the benchmark. The remaining challenge is therefore no longer benchmark design, but improving semantic assessment and report quality under the frozen protocol.

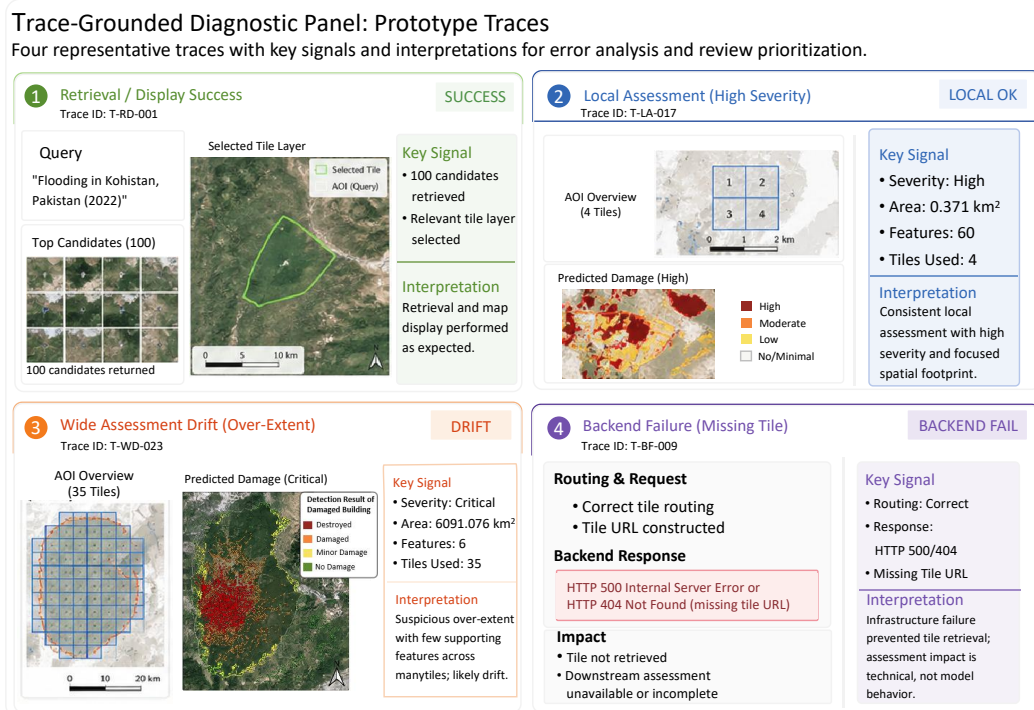


Figure 5: Trace-grounded diagnostic panel built from one retrieval-display trace, two completed assessment traces, and one model-detect failure. The table emphasizes measurable signals and failure attribution rather than a narrative workflow diagram.

Table 7: Split and hidden-test composition over hazard-event units. The counts are computed from the split manifest metadata; the final row records the hidden-test task set used for the evaluation tables.

Split	Units	Hazards	Paired	Area-only	Label-only
Train	212	20	29	182	1
Dev	46	15	2	42	2
Hidden test	46	15	5	39	2
Hidden-test tasks	262	6 fam.	—	—	—

H Reproducibility Protocol

This section records the reproducibility protocol used to bind each reported number to a fixed split, a fixed task subset, a saved prediction file, and a pre-declared statistical rule.

Stage 0: Freeze benchmark artifacts. Before any model comparison, freeze four benchmark files: (1) the event-level split manifest; (2) the finalized task records with task family, difficulty, and gold fields; (3) a hidden-test package with gold outputs removed; and (4) an evaluation configuration file containing metric thresholds, review thresholds, and the seed list. For assessment and report tasks, this freeze also includes the severity/evidence package described in Table 9: the severity label glossary, impact-fact format, evidence-key registry, and report template. In the repository these core files are already frozen, and adjudicated hidden-test overrides are maintained under the same controlled boundary. No prompts, routing rules, threshold values, or grading thresholds may change after this freeze point.

Stage 1: Validate the benchmark itself. Run benchmark-quality checks before system evaluation. Specifically, report split coverage by hazard and supervision type, dual-annotation agreement on a stratified subset, oracle execution validity for each task family, and human-reference solvability on a

Table 8: Task-family eligibility and hidden-test counts. The table records how each family is instantiated from the split manifest and frozen supervision artifacts.

Family	Minimum manifest-side requirement	Current hidden-test count	Primary frozen supervision
Event retrieval	Hazard-event unit assigned to a split	44 hidden-test tasks	canonical event identifier
Map grounding	Retrieval-eligible unit plus canonical displayable source or layer	44 hidden-test tasks	display source or layer target
Damage localization	<code>has_area</code> or <code>has_label</code>	44 hidden-test tasks	geometry target and threshold rule
Drone reinspection	<code>has_area</code> for ROI construction	42 hidden-test tasks	mission template and interrupt policy
Assessment synthesis	Localization-eligible unit plus severity target and evidence keys	44 hidden-test tasks	severity target plus evidence keys
Report generation	Assessment-eligible unit plus output schema	44 hidden-test tasks	report slot contract and grounding rule

Table 9: Current freeze-package status. Core policy files are frozen, and task-level hidden-test assessment/report overrides are maintained as adjudication-controlled benchmark artifacts.

Artifact	Schema fields	Freeze rule	Current workspace signal
Severity label glossary	<code>severity</code>	Lock label names and ordinals before distribution	5 labels frozen: none-critical
Impact fact format	<code>impact facts</code>	Store atomic, source-grounded facts only	Core format frozen
Evidence-key registry	<code>evidence keys</code>	Freeze a controlled key vocabulary and source types	11 evidence keys frozen
Report template	<code>report slots + metrics</code>	Freeze required slots and evidence-coverage rule	5 slots; grounding threshold 0.8
Assessment overrides	<code>severity + facts</code>	Live file should contain adjudicated rows only	44 hidden-test rows prepared for formal use
Report overrides	<code>facts + slots</code>	Live file should contain adjudicated rows only	44 hidden-test rows prepared for formal use

548 small but difficulty-balanced subset. If a task family fails these checks, revise the benchmark before
549 populating any system-comparison table.

550 **Stage 2: Tune once on development data.** Use only the public Event-IID development split
551 for prompt editing, workflow routing, review-threshold selection, and detector-mode selection.
552 Thresholds for review and reinspection should be selected to optimize a declared development
553 objective such as severity F1 at a false-autonomy cap or end-to-end score under a review-budget cap.
554 After tuning, freeze one configuration per system and reuse it unchanged for hidden-test and transfer
555 evaluation.

556 **Stage 3: Run the main Event-IID evaluation.** Evaluate the comparison systems on the Event-IID
557 hidden-test split summarized in Table 1. Deterministic systems may be run once; stochastic systems
558 should be run with three seeds using the same task order and tool budget. Save one prediction
559 file and one execution trace bundle per seed. The hidden-test scorer should emit per-task metrics,
560 family-level aggregates, end-to-end success, latency, tool calls, and a failure tag in {`success`,
561 `reasoning_failure`, `infra_failure`, `policy_review`}.

562 **Stage 4: Run transfer and orchestration robustness.** Populate Table 4 using the same frozen
563 system configurations. Evaluate the selected systems on hazard-held-out, region-held-out, and time-
564 held-out splits, then run a separate long-horizon subset containing tasks that require at least three

565 state-bearing stages such as retrieval→grounding→assessment or retrieval→localization→drone.
566 Report infrastructure-failure rate separately rather than folding it into task error.

567 **Stage 5: Run drone and review variants.** Populate Table 5 on the subset of hidden-test tasks that
568 are drone-eligible or review-eligible. Compare the full system against variants without the Drone
569 Agent, without review routing, and without both. Mission validity and ROI coverage should be
570 averaged only over tasks with a gold drone target; severity F1, ECE, review rate, and false-autonomy
571 rate should be computed over the full eligible subset.

572 **Stage 6: Run ablations.** Table 6 should use the Event-IID hidden-test split with the same frozen
573 prompts and thresholds as the main system. The purpose of this table is not to retune every ablation
574 independently, but to measure what breaks when key components are removed. This keeps the
575 comparison operationally honest.

576 **Per-table filling rules.** The paper can now be populated mechanically:

- 577 • **Table 1.** Use Event-IID hidden-test only; columns Retr., Ground., Local., Drone, Assess., and
578 Report should be family-level primary metrics, and E2E should be the percentage of tasks whose
579 primary criterion is met end to end.
- 580 • **Table 4.** Use hazard-held-out, region-held-out, time-held-out, and long-horizon subsets only. Do
581 not mix Event-IID numbers into this table.
- 582 • **Table 5.** Restrict mission validity and ROI coverage to drone-eligible tasks. Severity F1, ECE,
583 review rate, FAR, and E2E should use the full review/drone-eligible subset with frozen thresholds.
- 584 • **Table 6.** Use the same Event-IID hidden-test tasks as the main system row so that deltas remain
585 interpretable.

586 **Metric-to-family mapping.** Each task family should have one declared primary score used in the
587 family column of Table 1: event retrieval uses top-1 event accuracy; map grounding uses grounding
588 success against the correct source or layer; damage localization uses GeoJSON F1 or IoU with
589 thresholded partial credit; drone reinspection uses mission validity as the primary score and ROI
590 coverage as an auxiliary score; assessment synthesis uses severity F1; and report generation uses
591 report grounding score against required evidence keys. End-to-end success is defined per task record
592 by its primary metric and threshold.

593 **Required run artifacts.** Each evaluated run should save the following files in a stable directory
594 structure:

- 595 1. `config.json`: frozen prompt revision, tool budget, threshold values, and seed;
- 596 2. `predictions.jsonl`: one record per task with raw outputs and normalized predictions;
- 597 3. `trace.jsonl`: ordered tool calls, intermediate states, and timestamps;
- 598 4. `metrics.json`: aggregate metrics, calibration summaries, and failure counts;
- 599 5. `failures.csv`: task ID, failure type, and a short diagnostic note.

600 **Stable run-directory convention.** To keep table generation mechanical rather than notebook-
601 driven, we recommend a fixed directory layout of the form `runs/<subset>/<system>/<config_`
602 `id>/seed_<k>/`. Here `subset` is one of Event-IID hidden test, hazard-held-out, region-held-out,
603 time-held-out, long-horizon, or drone/review evaluation; `config_id` identifies the frozen prompt
604 and threshold package; and each seed directory contains exactly the five files listed above. Human-
605 reference runs may use `seed_human` in place of a numeric seed, but should still emit the same file
606 names so that table scripts do not need a separate code path.

607 **Table-to-subset mapping.** Once this layout is fixed, each table should draw from exactly one
608 family of run directories:

- 609 • **Table 1** reads only Event-IID hidden-test runs.
- 610 • **Table 4** reads only hazard-held-out, region-held-out, time-held-out, and long-horizon runs.

- **Table 5** reads only drone/review runs.
- **Table 6** reads only Event-IID hidden-test ablation runs produced under the same `config_id` as the full system.

Statistical reporting. For stochastic systems, compute the main score for each seed, pool task-level predictions across seeds only where the metric definition permits it, and report means with 95% bootstrap confidence intervals. Pairwise claims against the full DisasterAgent row should use paired bootstrap or approximate randomization over the same hidden-test tasks. The paper should avoid significance testing on development data because that split is used for tuning.

Interpreting repeated seeds. Several supplementary systems in the evaluation stack are near-deterministic under the frozen benchmark boundary. In particular, the heuristic single-agent and ablation baselines often differ across seeds only in continuous auxiliary quantities such as latency or calibration, while thresholded family metrics and end-to-end success remain identical. The full-DisasterAgent configuration is also relatively stable because routing, thresholds, and task order are locked and several expensive background branches are disabled for reproducible formal execution. Accordingly, repeated seeds in this study should be read primarily as a stability check on the frozen evaluation path rather than as an attempt to estimate large stochastic variance.

Artifact status. The repository contains a split manifest over 212/46/46 train/dev/hidden-test hazard-event units, a task schema with family and metric fields, trace summaries that expose both successful assessments and infrastructure failures, a frozen core-policy package, a hidden-test task package, and formal hidden-test run artifacts for the completed main-table systems. The package includes a frozen evaluation configuration, hidden-test manifest metadata, and 262 gold-removed task records. The freeze package includes severity labels, impact-fact schema, evidence-key registry, report-slot contract, and controlled task-level assessment/report overrides.

Supplementary experiment status. The main hidden-test comparison in Table 1 is complete under the frozen evaluation boundary, and the appendix now also includes populated long-horizon, drone/review, ablation, validation, and materialized held-out transfer diagnostics. The remaining caveat is therefore no longer missing rows, but *execution-mode comparability*: the held-out transfer rows use the fast bounded evaluation adapter for Full DisasterAgent after the non-fast bounded path timed out on the first hazard slice attempt. If later revisions replace those rows, they should do so under the same frozen scorer, manifests, and file contract so that the updated transfer numbers remain directly comparable to the current appendix tables.

I Task Record Summary

The task schema stores:

- task metadata: task ID, split, family, difficulty;
- execution constraints: allowed tools and tool budget;
- context: candidate events, pre/post assets, seed map source, region of interest, simulator start state;
- gold outputs: event ID, hazard type, map source, target region or GeoJSON, drone mission targets, allowed interrupts, minimum coverage, severity, evidence keys;
- evaluation settings: primary metric, auxiliary metrics, grading thresholds;
- provenance: event name, hazard type, region, source bundle, review requirement, operator-control level, annotation status.

J Artifact Summary

The package stack supporting this paper has three layers:

1. benchmark-definition artifacts: split metadata, task schema, hidden-test task package, and frozen evaluation configuration;

- 656 2. policy artifacts: severity labels, evidence-key registry, impact-fact schema, report template, and
657 task-level overrides;
- 658 3. execution artifacts: prediction files, trace bundles, aggregate metrics, and failure summaries for
659 formal runs.